REM Programmer's Guide

MyEmulator2 Linux on Termux and the Galaxy Fold

Revision 1.0 — 2026-09-22

Purpose and status

This guide is the practical companion to `MYEMULATOR2_ASSEMBLY_PROGRAMMING_MANUAL.md`. It explains how to use the 32-bit MyEmulator2 Linux environment deployed through Termux on a Galaxy Fold, and how to write programs for that environment.

The Android package and phone boot are not yet physically verified. Commands marked **phone test required** must be run on the Fold before they are considered successful. The host-side QEMU and Linux tests are separate from Android execution.

Feature status used in this guide:

Status	Meaning
Tested	Passed a repository regression or documented host test
Built, device test pending	Artifact exists but has not been run on the Fold
Planned/incomplete	Interfaces or implementation are not complete

1. Getting started on the Galaxy Fold

1.1 Transfer and install

On the Linux Mint desktop, connect the Fold with USB debugging enabled:

```
adb devices
adb push REM-android-arm64.zip /sdcard/Download/
```

In Termux:

```
termux-setup-storage
pkg update
pkg install bash coreutils tar
mkdir -p "$HOME/rem"
unzip "$HOME/storage/downloads/REM-android-arm64.zip" -d "$HOME/rem"
chmod 0755 "$HOME/rem/launch-rem.sh" \
  "$HOME/rem/qemu-system-myemulator32"
cd "$HOME/rem"
./launch-rem.sh
```

If the package was produced without bundled runtime libraries, install the Termux libraries requested by the package instructions:

```
pkg install glib zlib
```

The launcher does not require Android root, `proot`, or a graphical display. It runs the ARM64 QEMU executable directly and connects the guest serial console to the Termux terminal.

The package contains:

```
qemu-system-myemulator32   Android ARM64 REM QEMU
vmlinux                    REM Linux kernel
rootfs.ext4                persistent guest filesystem
launch-rem.sh              Termux launcher
lib/                       bundled libraries, when available
```

1.2 Launch, exit, and restart

Start the same persistent installation whenever REM is needed:

```
cd "$HOME/rem"
./launch-rem.sh
```

The guest should present its interactive serial shell. **Phone test required:** the first launch must be tested on the Fold; a successful host QEMU run does not prove Android execution.

Exit using the guest's normal shutdown/exit path. Do not force-stop Termux while the guest is writing `rootfs.ext4`. Then launch the same script again:

```
./launch-rem.sh
```

Do not delete, recreate, or overwrite `rootfs.ext4`: it is the persistent disk. A restart is a new QEMU process using the same file.

1.3 Verify persistence

Inside REM:

```
echo PHONE_PERSIST_TEST > /root/phone-persist.txt
cat /root/phone-persist.txt
```

Exit normally, start REM again, and run:

```
cat /root/phone-persist.txt
```

The file must still contain `PHONE_PERSIST_TEST`. Record the complete console output if the marker is missing, especially dynamic-loader, ext4, serial, or kernel-fault messages.

2. The 32-bit programming model

REM Linux runs on the MyEmulator2 target, not ARM, RISC-V, or x86. It is a little-endian, byte-addressed 32-bit CPU with fixed-width 32-bit instructions. Instructions are 4-byte aligned and normal sequential execution advances `PC` by four.

General registers are `r0`–`r15`:

Registers	Use
`r0` / `zero`	Hardwired zero
`r1`–`r4`	First four arguments and caller-saved temporaries
`r5`–`r12`	Callee-saved registers
`r13` / `sp`	Downward-growing, banked stack pointer
`r14` / `lr`	Link/return register
`r15`	Caller-saved temporary; GCC may use it as a frame base

The status register contains carry, signed overflow, interrupt priority, and supervisor mode. It does not have an ARM-style condition-code model. The complete instruction and encoding reference is in the companion manual.

2.1 Addressing

Memory operands use a base register and signed 13-bit displacement:

```
lw  r1, 0(r5)
sw  r1, 4(r5)
lbu r2, 1(r5)
```

There is no implicit post-increment addressing. Construct a full address with the relocation pair:

```
lui r5, %hi(buffer)
ori r5, r5, %lo(buffer)
```

Natural alignment is required for halfword and word accesses. Misaligned accesses raise a data-alignment exception.

2.2 Calls and stack frames

The current ABI passes arguments in `r1`–`r4`, then on the stack. A 32-bit return value is in `r1`; a 64-bit return uses `r1:r2`, low word first. The stack is downward-growing, 16-byte aligned at call boundaries, and has no red zone.

```
.text
.global add_two
add_two:
        add r1, r1, r2
        ret
```

A non-leaf function must preserve any incoming link value and all modified callee-saved registers:

```
function:
        addi sp, sp, -16
        sw   r5, 0(sp)
        sw   lr, 4(sp)
        # body, including nested calls
        lw   lr, 4(sp)
        lw   r5, 0(sp)
        addi sp, sp, 16
        ret
```

Structure-return rules, aggregate passing, variadic register-save areas, unwind metadata, dynamic TLS, and floating-point register conventions are not frozen. Treat compiler-generated code as the authority for those cases.

3. Programming in C

3.1 What is available

The repository contains:

- a bare-metal/static `myemulator2-elf` binutils toolchain;
- a GCC backend for integer and soft-float code;
- a hosted MyEmulator2 Linux/musl build path;
- minilibc and libc regression examples.

The native GCC runtime has been exercised in host QEMU fixtures, but the Android deployment package does not currently claim to contain GCC, headers, or a complete package manager. **Built, device test pending:** whether a future root filesystem includes a usable in-guest compiler must be verified on the Fold separately.

For host-side toolchain construction:

```
./toolchain/scripts/fetch-binutils.sh
./toolchain/scripts/build-binutils.sh
```

```
./toolchain/scripts/fetch-gcc.sh
PATH="$PWD/.toolchain-install/bin:$PATH" \
  ./toolchain/scripts/build-gcc.sh
```

The native Linux compiler path is distinct from Android's ARM64 QEMU build. Do not use the host desktop compiler to produce ordinary ARM binaries and expect them to run as MyEmulator2 guest programs.

3.2 C example

```c
#include <stdio.h>

int main(void)
{
        puts("Hello from REM");
        return 0;
}
```

Use the repository wrapper with the target compiler and musl prefix selected explicitly. This is the supported static-musl invocation:

```
MYEMU_TARGET_GCC=.toolchain-install/bin/myemulator2-elf-gcc \
MYEMU_MUSL_PREFIX=.musl-install \
toolchain/scripts/myemulator2-musl-gcc -O2 -static hello.c -o hello
```

If only the freestanding compiler is installed, use the repository's startup and test patterns instead:

```
myemulator2-elf-gcc -ffreestanding -nostdlib \
  -c program.c -o program.o
myemulator2-elf-gcc -ffreestanding -nostdlib \
  toolchain/examples/crt0.S program.o -lgcc -o program.elf
```

The working libc example is `toolchain/examples/linux-libc-regression.c`. It exercises allocation, strings, formatted output, file I/O, `uname`, directory creation, and sleep. It is evidence for the tested subset, not a promise that all libc/POSIX APIs are available.

4. Assembly programming

4.1 Instruction families

The current 32-bit instruction families are:

- integer ALU: `add`, `adc`, `sub`, `sbc`, `mul`, `mulh`, `mulhu`;
- division/remainder: `div`, `divu`, `rem`, `remu`;
- logic and shifts: `and`, `or`, `xor`, `not`, `sll`, `srl`, `sra`, `rol`, `ror`;
- comparisons: `seq`, `sne`, `slt`, `sge`, `sltu`, `sgeu`;
- immediate forms: `addi`, `subi`, `andi`, `ori`, `xori`, `slli`, `srli`, `srai`;
- memory: `lb`, `lbu`, `lh`, `lhu`, `lw`, `sb`, `sh`, `sw`;
- control transfer: `j`, `jal`, `jr`, `jalr`, `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`;
- system/MMU: `mfsr`, `mtsr`, `rfe`, `syscall`, `tlbflush`, `cas`, `break`, and `halt`.

Reserved opcodes and reserved fields are illegal. The authoritative complete encoding table is `docs/myemulator2-encoding.jso`

4.2 Direct Linux system calls

The Linux syscall instruction is `syscall`. The number is in `r1`, arguments are in `r2`–`r7`, and the return value replaces the number in `r1`. Linux errors are negative `-errno` values.

```
.text
.global _start
_start:
```

```
        li  r1, 64              # write
        li  r2, 1               # stdout
        lui r3, %hi(message)
        ori r3, r3, %lo(message)
        li  r4, message_end-message
        syscall 0
        li  r1, 94              # exit_group
        li  r2, 0
        syscall 0
1:      j 1b
.rodata
message: .ascii "Hello from assembly\n"
message_end:
```

The currently dispatched syscall subset includes `read` 63, `write` 64, `openat` 56, `close` 57, `lseek` 62, `fstat` 80, `mmap` 222, `munmap` 215, `brk` 214, `clone` 220, `execve` 221, `wait4` 260, `exit` 93, `exit_group` 94, `nanosleep` 101, `getpid` 172, `uname` 160, `pipe2` 59, `dup` 23, `dup3` 24, `fcntl` 25, `ioctl` 29, `writev` 66, and related process, directory, futex, and signal calls listed in the companion manual.

The syscall dispatcher is a maintained bootstrap implementation, not a complete generated Linux ABI. Verify availability in `linux/arch/myemulator2/kernel/syscall.c` before using a call.

5. Building software without a package manager

The guest root filesystem is persistent, but the Android package does not provide an ordinary Debian/Termux package manager inside REM. Keep source and build output under persistent directories such as `/root/src` and `/root/build`:

```
mkdir -p /root/src/hello /root/build
cd /root/src/hello
```

When `make` is available in the guest:

```
MYEMU_TARGET_GCC ?= .toolchain-install/bin/myemulator2-elf-gcc
MYEMU_MUSL_PREFIX ?= .musl-install
CC = MYEMU_TARGET_GCC=$(MYEMU_TARGET_GCC) \
    MYEMU_MUSL_PREFIX=$(MYEMU_MUSL_PREFIX) \
    toolchain/scripts/myemulator2-musl-gcc
CFLAGS = -O2

hello: hello.c
    $(CC) $(CFLAGS) -static -o $@ $<

clean:
    rm -f hello
```

If Make or the compiler is absent, build on the Linux desktop with the MyEmulator2 cross-toolchain and transfer the resulting source or executable. Do not copy a host x86-64 or Android ARM64 executable into the guest.

Install a self-built program in a persistent directory:

```
mkdir -p /root/bin
cp hello /root/bin/
chmod 0755 /root/bin/hello
/root/bin/hello
```

Planned/incomplete: a complete in-guest native development environment, package manager, and standard library package set.

6. Filesystem and file transfer

`rootfs.ext4` is the guest's persistent block device. Guest-created files survive QEMU restarts when the same image is reused. The Android launcher passes it read/write through the REM-specific `myemulator2-disk` backend.

For desktop-to-guest transfer, use one of these practical paths:

1. Transfer source or binaries to Android with `adb push`, then use a future shared-directory mechanism if enabled.
2. Use Termux storage and copy files into the deployment directory before launching; the guest then needs a supported transfer path.
3. Transfer source through the guest's implemented networking stack when that machine support exists.

The current package explicitly guarantees the QEMU launcher and persistent disk, not a host directory mount or a complete Android/guest file-sharing protocol. Do not claim networking or shared folders until tested.

Always exit cleanly before copying or replacing `rootfs.ext4`. Keep backups outside the active deployment directory when experimenting.

7. Networking

Networking is **planned/incomplete** for the current Android deployment. Sockets are not a promise merely because Linux has socket syscall numbers on other architectures. The present REM machine focuses on the serial console, timer, block device, ELF execution, and ext4 userspace.

When networking is implemented, document the QEMU NIC model, guest interface name, IP configuration, Android Termux permissions, and persistence/security behavior before relying on it. Until then, use `adb`, Termux storage, or serial-console workflows for transfer.

8. Debugging

8.1 First checks

For a program that does not start:

```
file ./program
readelf -h -l ./program
readelf -r ./program
```

Confirm that it is an ELF32 MyEmulator2 executable, not an ARM or host ELF, and that the entry point and load segments match the current linker script.

For a shell or filesystem failure, capture:

```
uname -a
ls -l /bin/busybox /sbin/init /bin/sh
cat /proc/mounts
```

Then record the complete QEMU console output from boot to failure.

8.2 Assembly and CPU faults

Record the fault vector/cause, `PC`, `SR`, `SP`, `LR`, and faulting address. Check for:

- an odd or otherwise misaligned instruction target;
- a reserved opcode or nonzero reserved encoding field;
- an unaligned `LH`, `LW`, `SH`, or `SW`;
- a bad signed displacement;
- a missing `%hi`/`%lo` relocation pair;

- a syscall number or argument placed in the wrong register;
- failure to preserve `r5`–`r12`, `sp`, or `lr`;
- a stack frame that is not 16-byte aligned.

The tracked CPU, integer, system, ISA, binutils, GCC, and Linux regression scripts are the preferred diagnostic fixtures. A host QEMU failure should not be described as an Android failure, and an Android dynamic-linker failure should not be attributed to the REM CPU without evidence.

8.3 Known limitations

The following are known incomplete areas:

- Android phone boot has not yet been verified.
- The complete Linux syscall table is not generated.
- Dynamic linking, shared libraries, GOT/PLT, and dynamic TLS are incomplete.
- Full signals and stable signal-frame ABI are incomplete.
- Aggregate/variadic calling conventions are not frozen.
- Complete libc/POSIX coverage is not verified.
- Networking and a guest package manager are not complete.

9. Status matrix

Capability	Status
MyEmulator2 32-bit ISA/QEMU execution	**Tested** by repository CPU/system tests
ELF32 static assembly/link workflow	**Tested** by binutils tests
GCC integer/soft-float backend	**Tested** by GCC/toolchain fixtures
Linux user ELF startup and process lifecycle	**Tested** in host QEMU
Serial console and interactive ext4 shell	**Tested** in the documented host milestone; replacement deployment image and final Android package must be revalidated
Android ARM64 QEMU executable	**Built, device test pending**
Galaxy Fold boot	**Phone test required**
Persistent ext4 restart on Fold	**Phone test required**
Complete in-guest GCC/Make environment	**Built/incomplete; device test pending**
Networking	**Planned/incomplete**
Dynamic linking and full libc	**Planned/incomplete**

Appendix A. Source map

- Architecture and encoding: `docs/MYEMULATOR_2_ARCHITECTURE.md`, `docs/myemulator2-encoding.json`
- ABI and relocations: `docs/MYEMULATOR_2_ABI.md`
- Linux machine contract: `docs/MYEMULATOR_2_LINUX_PORT.md`
- Assembly/toolchain workflow: `toolchain/README.md`
- GCC backend: `toolchain/gcc/myemulator2.md`
- User linker: `toolchain/userspace/myemulator2-user.ld`
- Linux syscalls: `linux/arch/myemulator2/kernel/syscall.c`
- Android install: `android/TERMUX_DEPLOYMENT.md`
- Direct assembly example: `toolchain/examples/linux-echo.S`
- C/libc example: `toolchain/examples/linux-libc-regression.c`
- Host Linux regressions: `toolchain/scripts/test-linux-*.py`